5 - 17 marzo 2026

Wireshark: programma che serve per vedere il traffico di rete

abbiamo fatto delle prove in localhost praticamente abbiamo aperto un pagina web e abbiamo analizzato tutte le informazioni che vengono inviate al login (con username e password) vediamo che il server risponde con il Token di sessione, rubare il token di sessione è alla base del phishing

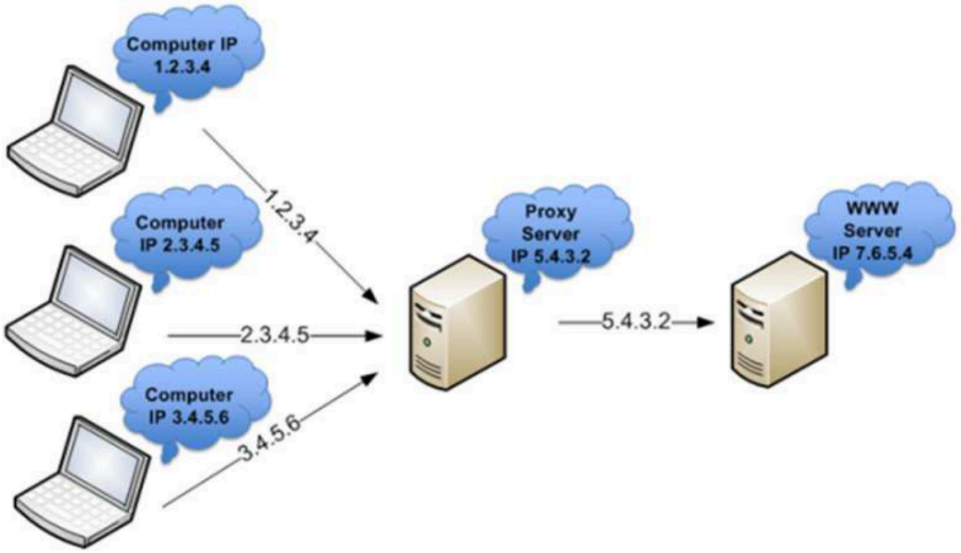
Ci sono due modi per risolvere questo problema:

- Usare HTTPS
- Refreshare il token di sessione spesso (così anche se rubato si ha poco tempo per usarlo)

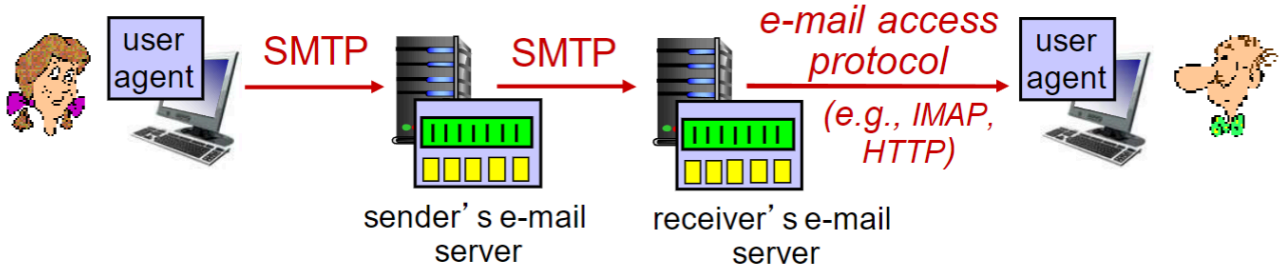
Proxy Server
serve per mascherare l'ip di un servizio sulla rete, è un server intermedio che permette di controllare, filtrare e mascherare il traffico da parte di uno o più client. Nello specifico:

1. I client mandano le richieste al proxy.
2. Il proxy le invia al server destinazione.
3. Il server riceve la richiesta del proxy.
4. Il server invia la risposta al proxy.
5. Il proxy invia la risposta del server al client originale.

I proxy sono utili anche per monitorare il traffico di rete da parte degli host che lo usano.
Avendo una cache interna alcune volte può rispondere con una pagina vecchia



SMTP(Simple Mail Transfer Protocol): questo è il protocollo utilizzato per lo scambio di posta elettronica, in specifico si occupa di trasferire le mail dal server mittente a quello destinazione. Usa la porta 25 su TCP



Noi quindi usiamo SMTP per mandare la mail al nostro server di posta che (anche lui usando SMTP) la invierà al server di posta del destinatario che leggera questa mail usando o POP o IMAP

La @ nella mail viene inserita per distinguere il nome utente dal server di destinazione
Di seguito quello che accade veramente:

```
S: 220 BBN-UNIX.ARPA Simple Mail Transfer Service Ready
C: HELO USC-ISIF.ARPA
S: 250 Hello BBN-UNIX.ARPA, pleased to meet you
C: MAIL FROM:<Smith@USC-ISIF.ARPA>
S: 250 OK
C: RCPT TO:<Jones@BBN-UNIX.ARPA>
S: 250 OK
C: RCPT TO:<Green@BBN-UNIX.ARPA>
S: 550 No such user here
C: RCPT TO:<Brown@BBN-UNIX.ARPA>
S: 250 OK
```

L'immagine mostra perfettamente la natura "botta e risposta" del protocollo SMTP, basato su comandi testuali chiari (come `HELO`, `MAIL FROM`, `RCPT TO`) e codici di stato standardizzati. In questo scenario, l'email è stata autorizzata per Jones e Brown, ma la consegna a Green è fallita a causa di un indirizzo inesistente.

Il client che si connette al server viene chiamato **user-agent**: si collega al server di posta e ci permette di interagire con le nostre mail, questo può essere fatto anche tramite SMPT ma è molto più conveniente usare **POP3** e **IMAP**

POP3(Post Office Protocol): scarica le mail dal server e ci permette la visualizzazione anche offline (cancella le mail dal server di posta)
IMAP: ci permette la visualizzazione delle mail senza scaricarle, è più semplice ma non ci permette di visualizzare le mail se siamo offline, usando questo protocollo ovviamente le mail non vengono cancellate dal server di posta (Google ha tutte le nostre email nei suoi database)

Caratteristica	POP3	IMAP
Dove è definito il protocollo?	RFC 1939	RFC 2060
Quale porta TCP viene utilizzata?	110	143
Dove sono memorizzate le e-mail?	PC dell'utente	Server
Come vengono lette le e-mail?	Offline	Online
Tempo di connessione richiesto?	Poco	Molto
Uso delle risorse del server?	Minimo	Esteso
Caselle di posta multiple?	No	Sì
Chi esegue il backup delle caselle di posta?	Utente	ISP (Provider)
Adatto per utenti mobile?	No	Sì
Controllo utente sui download?	Poco	Elevato
Download parziale dei messaggi?	No	Sì
Le quote disco sono un problema?	No	Potrebbero diventarlo nel tempo
Semplice da implementare?	Sì	No
Supporto diffuso?	Sì	In crescita

Mail di SPAM: lo spam di mail nasce dalla famosa azienda spam che un giorno per pubblicizzare il suo prodotto ha inviato un sacco di mail contemporaneamente.

DNS (Domain Name System)
Gli indirizzi IP identificano in maniera univoca gli host in rete e la loro locazione: la struttura di un indirizzo IP, fornisce infatti informazioni sulla struttura della rete e la

posizione dell'host all'interno di essa. Gli IP non sono facili da ricordare, per un umano, in quanto costituiti esclusivamente da caratteri numerici.
Gli hostname nascono con lo scopo di associare un nome a un IP: diventa però necessario introdurre un meccanismo che permetta la traduzione degli hostname in indirizzi IP. Nasce così il DNS

l'associazione nome-indirizzoIP *(hostname)* viene risolta usando il protocollo DNS.

```
151.97.240.18        www.dmi.unict.it
151.97.240.4         www.unict.it
151.97.6.236         www.ing.unict.it
151.97.252.132       galileo.dmi.unict.it
```

Una panoramica sui dominii

1. Root Domain (.)

È il livello più alto della gerarchia.
Non si vede normalmente negli indirizzi (è implicito alla fine, es: *google.com.*).

2. Top-Level Domain (TLD)

Sono i domini di primo livello e si trovano subito sotto il root.
Esempi: .com .it .us .edu ecc...

3. Second-Level Domain (SLD)

È solitamente il nome scelto dall'organizzazione o dal proprietario del sito.

Esempi:

- *google.com* → "google" è il secondo livello
- *unict.it* → "unict" è il secondo livello

4. Sottodomini (Third-Level e oltre)

Sono livelli aggiuntivi sotto il secondo livello.

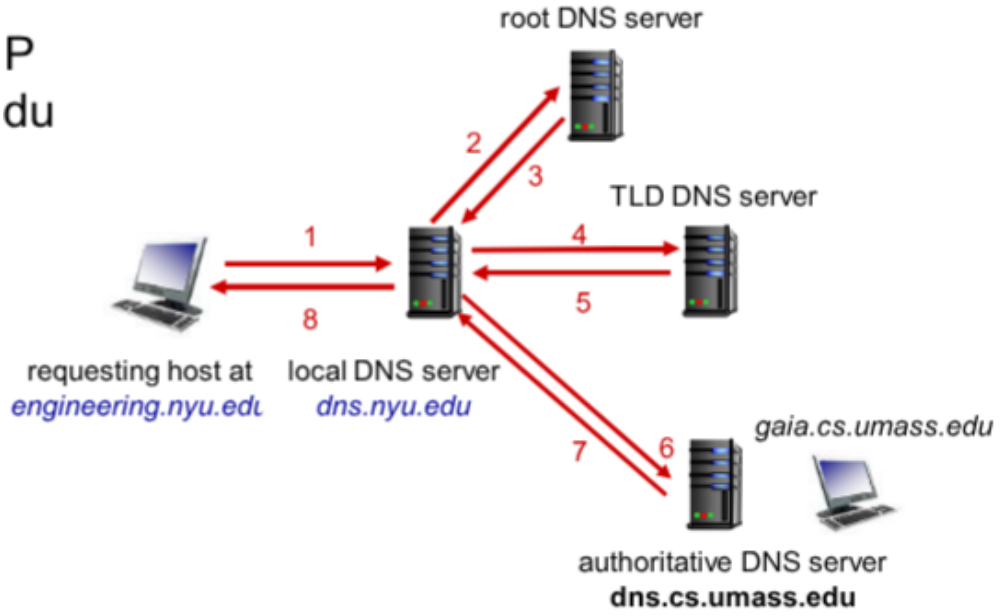
Esempi:

- `www.google.com`
- `mail.google.com`
- `api.service.example.com`

web.dmi.unict.it.
Qui abbiamo il . che è root, .it che è TDL, unict è SDL, e poi abbiamo i third-level e oltre

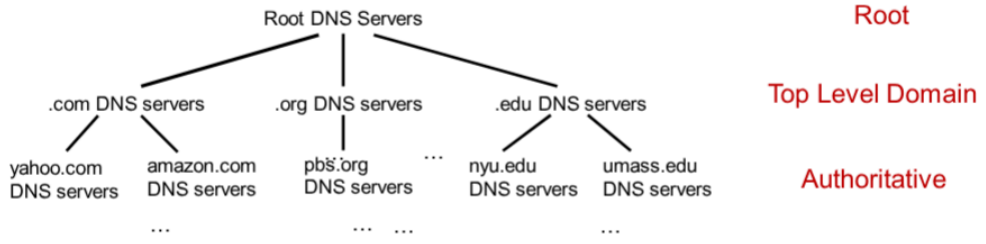
(.dmi e web)

Funzionamento DNS



Questi sono i passaggi che fa un dispositivo per risolvere un nome in un indirizzo IP.
Il client manda una richiesta al suo DNS server locale con l'indirizzo da cercare, il DNS server locale non sa direttamente dove trovare questo indirizzo quindi si appoggia a altri DNS server, inizia quindi dal *root DNS server*, che contiene e risponde con l'indirizzo del server da raggiungere per risolvere gli indirizzi *Top-Level Domain* (ovvero .it .us .fr .com .org ecc...), quindi il DNS server locale si rivolge al *TDL DNS server* che contiene e risponde con il dominio di secondo livello (.youtube .netflix .unict ecc...) da qui il DNS locale si rivolge all'ultimo DNS del percorso chiamato *authoritative DNS server* che contiene l'ultima associazione indirizzo IP-dominio

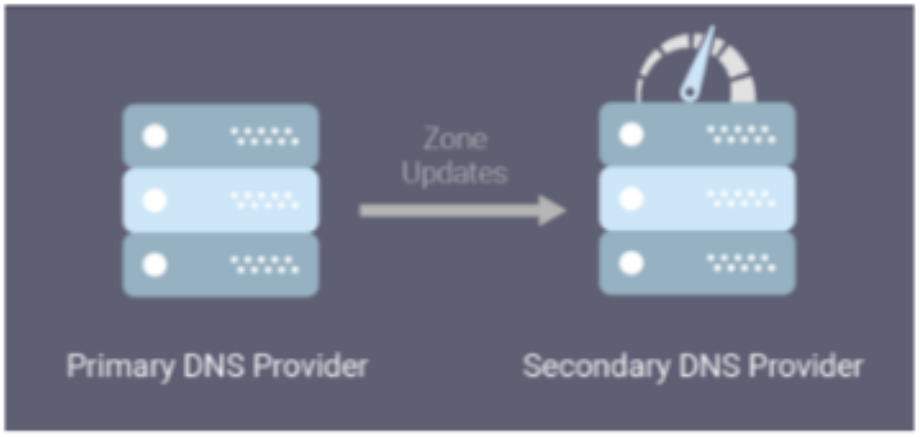
In pratica il DNS è un database distribuito in tutto il mondo, viene gestito come un albero, questo albero ha 13 radici e non una, questo è dovuto al fatto che esistono *13 DNS server root*, anche se ovviamente sono distribuiti in centinaia di macchine fisiche.
In pratica ogni dispositivo sa raggiungere un server che conosce più indirizzi di lui se l'indirizzo cercato non viene trovato(fino ad arrivare al server destinazione).



è un protocollo molto scalabile, in quanto la maggior parte delle richieste vengono risolte a livello basso (grazie alle cache) e quasi mai al root level. Banalmente se noi cerchiamo google.it su rete UNICT il server dell'ateneo sa risolvere l'indirizzo IP a livello 1 perché è salvato sulla sua cache di sistema essendo un link molto usato.

Esistono due tipi di server DNS in un'azienda:

- **Primario**: Qui risiedono i dati ufficiali (l'admin modifica qui i record del DNS DB)
- **Secondario**: Se il server primario dovesse andare offline, il server secondario continuerà a rispondere alle richieste degli utenti, garantendo che il sito web rimanga raggiungibile.
  Il processo di aggiornamento è semplice:
- Aggiorno i record nel Primario
- Vengono aggiornati in automatico anche i record nel secondario



Fuori contesto: posso mettermi in mezzo alla comunicazione tra due PC diventando un *man in the middle* e riuscendo a catturare tutti i dati che vanno da A a B

Esistono anche dei protocolli utilizzati per la diagnostica, uno di quelli più utilizzati è **SNPM (Simple Network Management Protocol)**, non ci interessa in modo reale come funziona, dobbiamo solo sapere che esiste e per cosa viene utilizzato (ha solo detto il nome praticamente).

Siamo passati a 3 - Transport_N.pdf

Nello stack abbiamo diversi layer:

- **Datalink layer**: comunicazione fisica tra gli host
- **Network layer**: comunicazione logica tra gli host
- **Trasport layer**: comunicazione logica tra i processi (usando IP e porta)

Il trasporto a livello trasport viene fatto usando:

- il **multiplexing** (sorgente): è il processo con cui un host incapsula i frammenti di dati provenienti da uno o più sockets in segmenti del transport layer utilizzando opportuni header.

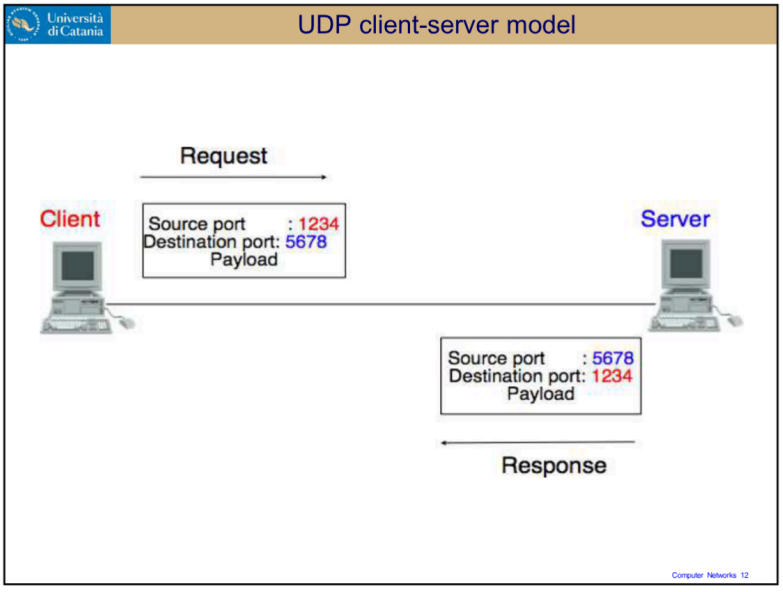
- il **demultiplexing** (destinazione): Consiste nella consegna dei segmenti al socket corretto e quindi al processo corretto.

Per fare queste cose vengono utilizzati due protocolli (uno o l'altro):

- **RDT(Reliable Data Transfer)**: affidabile e lento, **TCP**(Trasmission Control Protocol) è la sua implementazione più famosa. Viene diviso in diverse versioni dal nostro libro per farci capire come si è arrivati storicamente a TCP, esiste una singola versione.
- **UDP**: inaffidabile ma molto veloce

UDP(User Datagram Protocol): offre un servizio "best effort". Visto che non garantisce l'arrivo dei dati è un protocollo molto semplice, non viene stabilita una connessione reale tra sorgente e destinazione, grazie a queste caratteristiche è molto veloce.

- UDP è di tipo *connectionless*
- L'header del protocollo UDP occupa 8 byte. 2 byte per la porta del mittente, 2 per la porta del destinatario, 2 per la lunghezza del messaggio (header UDP + dati) e 2 per il checksum. Per confronto l'header del protocollo TCP occupa dai 20 ai 60 byte:
- E' un meccanismo che consente di rilevare errori di trasmissione. Questi errori spesso consistono in bit-flip, che possono stravolgere del tutto il significato dei dati trasmessi. Detto ciò, la verifica consiste nel calcolo di un checksum, in italiano somma di controllo, sia al mittente (che la inserirà nel frame) che al destinatario. La somma è calcolata in virtù dei dati contenuti nel frame. Se le due somme non dovessero coincidere, il pacchetto sarà da considerarsi fallato, e verrà scartato (questo Checksum vale anche per TCP).



Il canale di comunicazione però è per sua stessa natura fisica inaffidabile, quindi certi pacchetti possono andare persi, ecco che si ricorre a RDT per una comunicazione affidabile

RDT

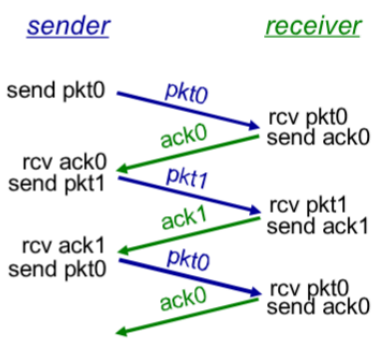
Riesce a rendere affidabile qualcosa di inaffidabile, lo fa pagando in termini di velocità. Il nostro libro ha creato delle versioni per farci capire come si è arrivati alla versione finale:

	Errors on forward channel	Errors on backward channel	ACK / NAK	Packet loss
RDT 1.0	**no**	**no**	-	no
RDT 2.0	**yes**	no	ACK / NAK	no
RDT 2.1	yes	**yes**	ACK / NAK	no
RDT 2.2	yes	yes	**ACK**	no
RDT 3.0	yes	yes	ACK	**yes**

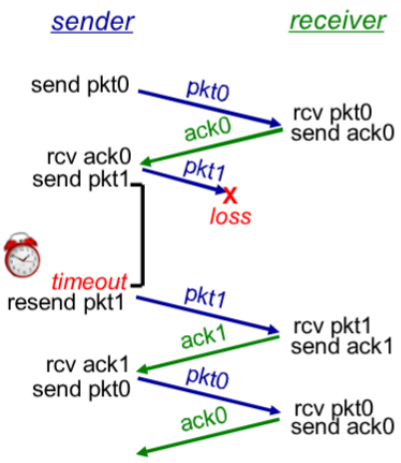
Osserviamo in breve le caratteristiche dei vari RDT prima di studiarne le FSM al dettaglio:

- *1.0.*: Non include nessun meccanismo effettivo, ma fa da base per le versioni successive.
- *2.0.*: Gestisce i pacchetti corrotti tramite segnali di ACK/NAK, ma non gestisce ACK e NAK corrotti.
- *2.1.*: Gestisce gli errori sul backward channel (un canale usato per l'invio di ACK/NAK).
- *2.2.*: Si elimina il simbolo dedicato NAK, utilizzando in maniera differente gli ACK (ottimizzazione)
- *3.0.*: Tutte le precedenti versioni gestiscono solo pacchetti corrotti. Con questa versione, si verifica anche la perdita dei pacchetti, introducendo il meccanismo di time-out.

Di seguito la versione 3



(a) no loss



(b) packet loss